North South University

Department of Electrical & Computer Engineering

Project Report

Course Code:	CSE323
Section:	14
Course Name:	Operating Systems Design

Name:	Reefa Shehnil
NSU ID:	2231539042
Email:	reefa.shehnil@northsouth.edu

Date of Submission:	26h December,2025

Visual Deadlock

Introduction

An Operating System (OS) is responsible for managing system resources and ensuring that multiple processes can execute safely and efficiently. One of the most critical problems in resource management is deadlock, a situation where a set of processes are blocked forever because each process is holding a resource and waiting for another resource held by another process.

The goal of this project is to design and implement a visual Resource Allocation and Deadlock Simulator that helps users understand how deadlocks occur, how they are detected, and how they can be resolved using preemption. Unlike traditional command-line simulations, this project uses a Graphical User Interface (GUI) to visually represent processes, resources, allocations, requests, waiting states, and deadlock resolution through animated arrows.

This simulator is mainly intended as an educational tool for Operating Systems students to clearly visualize complex concepts that are often difficult to understand through theory alone.

1. Requirements Analysis

Functional Requirements

The system must:

- **Allow users to create multiple processes dynamically.**
- **Allow processes to request system resources.**
- **Visually show allocated resources using arrows.**
- **Visually show waiting (request) states using dashed arrows.**
- **Detect deadlock situations automatically.**
- **Ask the user whether to resolve deadlock using preemption.**
- **Select a victim process and release its resources.**
- **Reallocate freed resources to waiting processes.**

- **Allow modification of existing processes (request or release resources).**
- **Provide clear visual feedback for every operation.**

Non-Functional Requirements

The system should:

- **Have an intuitive and user-friendly GUI.**
- **Use smooth and understandable animations.**
- **Be responsive and stable during execution.**
- **Be easy to extend for future features.**
- **Run on standard systems without special hardware requirements.**

Tools & Technologies Used

- **Python – Core programming language**
- **Tkinter – GUI framework**
- **Matplotlib – Graph visualization and animations**
- **Object-Oriented Programming (OOP) – Code organization**

2. Implementation (Methodology)

The system is implemented using a modular and object-oriented approach, with clear separation between logic, visualization, and user interaction.

System Architecture

The application is built around a main class:

- **DeadlockSimulatorApp – Handles GUI, user interaction, visualization, and system logic.**

Data Structures

- **RESOURCES: Dictionary storing total instances of each resource.**
- **processes: Dictionary storing allocated resources for each process.**
- **waiting_requests: Dictionary storing pending resource requests.**

- **pos: Stores graphical positions of processes and resources.**
- **deadlock_flag: Indicates whether a deadlock exists.**

Graph Visualization

- **Resources are drawn as rounded rectangles.**
- **Processes are drawn as circles.**
- **Allocated resources are shown using solid orange arrows.**
- **Waiting requests are shown using dashed green arrows.**
- **Deadlock is visually highlighted with a warning message.**

Resource Request Handling

1. **User selects a process.**
2. **User inputs resource requests through a GUI dialog.**
3. **Available resources are checked.**
4. **If resources are available → allocation occurs.**
5. **If not → request is placed in waiting state.**
6. **Animations show each step clearly.**

Deadlock Detection

A deadlock is detected when:

- **A process is waiting for resources.**
- **No further allocation is possible.**
- **Circular waiting exists implicitly.**

Preemption Strategy

- **When deadlock is detected, the user is prompted.**
- **A victim process is selected (process holding the most resources).**
- **Resources of the victim are released.**
- **Waiting processes are rechecked and allocated resources if possible.**

- **Animations show the entire resolution process.**

3. Testing and Validation

Test Cases

1. **Single Process Allocation**
    - **Expected: Resources allocated successfully.**
    - **Result: Passed.**
2. **Multiple Processes with No Deadlock**
    - **Expected: All processes complete allocation.**
    - **Result: Passed.**
3. **Deadlock Creation**
    - **Expected: Deadlock warning displayed.**
    - **Result: Passed.**
4. **Preemption Execution**
    - **Expected: Victim selected, resources released, deadlock resolved.**
    - **Result: Passed.**
5. **Release Resources**
    - **Expected: Waiting processes get resources.**
    - **Result: Passed.**

Validation

- **GUI updates correctly after every operation.**
- **Resource counts remain consistent.**
- **No negative resource allocation occurs.**
- **System remains stable during repeated operations.**

4. Challenges Faced

Situation

The simulator required real-time visualization of resource allocation, waiting states, and deadlock resolution, which is complex to represent clearly.

Task

To design a system that visually represents all OS concepts accurately while keeping animations understandable and smooth.

Action

- **Used Matplotlib for precise control over arrows and shapes.**
- **Introduced step-by-step arrow animations.**
- **Carefully calculated positions to avoid overlapping visuals.**
- **Integrated GUI dialogs instead of console input.**

Result

The simulator clearly shows:

- **Allocation flow**
- **Waiting conditions**
- **Deadlock occurrence**
- **Deadlock resolution**
  This significantly improves understanding for users.

5. How the Challenges Were Solved

Situation

Deadlock resolution through preemption needed to be intuitive and not confusing for users.

Task

To design a preemption mechanism that:

- **Is fair**
- **Is visual**
- **Clearly explains what is happening**

Action

- **Implemented a victim selection strategy based on maximum resource holding.**
- **Added animated arrows for resource release.**
- **Used dialog boxes to involve the user in decision-making.**
- **Updated waiting queues automatically after preemption.**

Result

Users can:

- **See exactly which process is preempted.**
- **Observe resources being released.**
- **Understand how deadlock is resolved step-by-step.**

Conclusion

This project successfully demonstrates the concepts of resource allocation, deadlock detection, and deadlock recovery using a visually rich and interactive GUI-based simulator. By combining Operating System theory with graphical visualization, the simulator makes abstract concepts easier to understand and more engaging for students.

The use of Python, Tkinter, and Matplotlib allowed rapid development while maintaining clarity and flexibility. Features such as animated arrows, user-controlled preemption, and real-time visualization make this simulator an effective educational tool.

Overall, this project fulfills its objectives and provides a strong foundation for further enhancements such as Banker's Algorithm, deadlock prevention techniques, or step-by-step execution modes.